

Ergebnisbericht: Abgabe 2 - Betriebssysteme VL Uni Trier WS 25/26

Aufgabe 1: Semaphore-basierte Implementierung

Im ersten Teil der Aufgabe wurde das Problem als Multi-Threaded-Anwendung in **Python** modelliert, wobei die Synchronisation ausschließlich über Semaphore erfolgt.

1.0 How to run

Die Simulation kann durch Ausführen der Datei `santa_semaphore.py` gestartet werden. Vorher können die Parameter für die Anzahl der Rentiere (r), Elfen (e) und die erforderliche Gruppengröße hilfsbedürftiger Elfen (p) direkt im Script angepasst werden.

```
python santa_semaphore.py
```

1.1 Thread-Modellierung

Jeder Beteiligte (1 Weihnachtsmann, r Rentiere, e Elfen) wurde als eigenständiger Thread realisiert, was - wie gefordert - mindestens $1+r+e$ Threads ergibt. In Python habe ich hierfür das `threading` -Modul verwendet.

1.2 Synchronisationsmechanismen

Folgende Logik wurden implementiert:

- `santa_sem` : Ein Semaphore, an dem der Weihnachtsmann-Thread blockiert, bis er von der Gruppe hilfsbedürftiger Elfen oder den angereisten Rentieren geweckt wird.
- `reindeer_sem` : Ein Semaphore, an dem die Rentiere warten, bis sie in voller Mannes- bzw. Rentierstärke am Nordpol angekommen sind und der Weihnachtsmann den Schlitten bereit macht.
- `elf_sem` : Ein Semaphore, der sicherstellt, dass die Gruppe von p Elfen nacheinander Hilfe erhält.
- `mutex` : Ein gegenseitiger Ausschluss-Semaphore zum Schutz der globalen Zählervariablen (`elf_count`, `reindeer_count`).

Die Rentiere warten mittels `time.sleep(random.uniform(8, 10))`, um ihre Ankunftszeit zu simulieren. Sobald das letzte Rentier eintrifft, wird der Weihnachtsmann geweckt. Ähnlich verfahren die Elfen, die nach einer zufälligen Produktionszeit von `time.sleep(random.uniform(1, 5))` Hilfe benötigen.

1.3 Logischer Ablauf und Priorisierung

Ein kritischer Aspekt ist die Priorisierung der Rentiere gegenüber den Elfen. Wenn der Weihnachtsmann geweckt wird, prüft er zunächst den Status der Rentiere. Sind alle r Tiere anwesend, wird die Auslieferung gestartet. Erst wenn dies nicht der Fall ist, widmet er sich der wartenden Elfen-Gruppe. Dies verhindert, dass die zeitkritische Geschenkauslieferung am 24. Dezember durch Produktionsfragen verzögert wird.

Aufgabe 2: Implementierung mittels Docker und ZeroMQ

Der zweite Teil der Aufgabe transformiert das Problem in ein verteiltes System, bei dem die Akteure in isolierten Docker-Containern laufen und innerhalb eines Netzwerk kommunizieren.

2.0 How to run

Gemäß den Anforderungen kann die Simulation der Aufgabe 2 wie folgt gestartet werden:

1. **Voraussetzungen:** Installiertes Docker und Docker Compose.
2. **Starten:** Wechseln Sie in das Verzeichnis mit der `docker-compose.yml` und führen Sie `docker compose up --build` aus. Dies baut die Images und startet alle Beteiligten gleichzeitig.
3. **Überwachung:** Die Ausgaben aller Container werden farblich markiert im Terminal ausgegeben. Sie können beobachten, wie Rentiere eintreffen und Elfen Probleme melden.
4. **Beenden:** Die Simulation kann mittels `Strg+C` gestoppt und mit `docker compose down` vollständig bereinigt werden.

2.1 Architektur des verteilten Systems

Jeder Akteur (Santa, Elfen, Rentiere) wird als separater Docker-Container ausgeführt. Dies erfordert eine Abkehr vom Shared-Memory-Modell hin zum Message-Passing-Modell.

- **Santa-Zentrale:** Der Weihnachtsmann-Container fungiert als zentraler Server. Er verwaltet intern die Zählerstände und entscheidet basierend auf den eingehenden Nachrichten, wann welche Aktion ausgelöst wird.
- **Akteur-Clients:** Elfen und Rentiere senden ihren Status an die Zentrale und warten auf eine Bestätigung (Reply). Der Unterschied zwischen den Akteuren wird über Umgebungsvariablen (`ROLE`) gesteuert, die beim Start des Containers gesetzt werden.

Die Container nutzen als Base-Image `python:3.11-slim`. Der Quellcode wird beim Build in die Container kopiert, und die notwendigen Python-Abhängigkeiten (`pyzmq`) werden über `RUN pip install pyzmq` installiert.

2.2 Kommunikation mit ZeroMQ

Die Synchronisation erfolgt nachrichtenbasiert mittels ZeroMQ.

- **Muster:** Es wurde das `ROUTER/REQ` -Pattern gewählt. Der `ROUTER` -Socket bei Santa erlaubt es, Nachrichten von vielen anonymen Clients (Elfen/Rentiere) zu empfangen und die Antworten gezielt an die richtigen Absender zurückzusenden.
- **Nachrichtenfluss:** Wenn ein Elf ein Problem hat, sendet er ein `ELF_PROBLEM`. Santa speichert die Identität des Absenders in einer Warteschlange. Erst wenn die Gruppe die benötigte Anzahl `$p$` erreicht, sendet Santa die `PROBLEM_SOLVED` -Bestätigung an alle `$p$` Elfen zurück.

2.3 Container-Orchestrierung

Um die Simulation mit über 20 Containern handhabbar zu machen, habe ich wie vorgeschlagen Docker Compose eingesetzt.

- **Automatisierung:** Ein einfacher Aufruf von `docker compose up` startet die gesamte Infrastruktur.
- **Skalierbarkeit:** Über das `replicas` -Feld in der Konfiguration `docker-compose.yml` kann die Anzahl der Elfen (`$e$`) und Rentiere (`$r$`) dynamisch angepasst werden, ohne den Code ändern zu müssen.

3. Vergleich der Ansätze

Die beiden Implementierungen zeigen die Evolution von lokaler Thread-Synchronisation zu Cloud-nativen verteilten Systemen.

Merkmal	Aufgabe 1 (Semaphore)	Aufgabe 2 (Docker/ZMQ)
Kommunikation	Gemeinsamer Speicher (Shared Memory)	Nachrichtenbasiert (Network/ZMQ)
Synchronisation	Hardware-nahe Semaphore	Logische Blockierung durch Request/Reply
Skalierbarkeit	Limitiert auf einen Prozess/Host	Horizontal skalierbar über mehrere Hosts (mit Kubernetes)
Isolierung	Gering (ein Fehler kann Prozess stoppen)	Hoch (Container-Isolierung)

Besonders hervorzuheben ist, dass die ZeroMQ-Lösung inhärent vor Race Conditions bei den Zählern schützt, da nur der Santa-Zentral-Thread Zugriff auf die internen Listen hat. In der Semaphor-Lösung muss dies explizit durch einen Mutex abgesichert werden.